PACE: Predictive Auto-Complete Engine using Hashing and BST.

Author: Shaikh Aakef

Under the guidance of: Ms. Mayuri Fakirpure

Department of AI and Machine Learning

Symbiosis Institute of Technology, Pune 412115

Introduction:

In the continuous evolving digital world, predictive text and auto-complete engines are no longer just mere user-friendly software benefits. They have become part of our instinctive nature while interacting on any platform. From Google searches and WhatsApp chats to Integrated Development Environments (IDEs), these systems process natural language in real-time to anticipate user intent, reduce manual efforts, and minimize human typographical errors. However, beneath the seamless user experience lies a complex computing challenge: the ability to instantly parse, retrieve, and rank massive datasets of vocabulary within milliseconds.

Problem Statement:

Everyday as we rely upon our mobile devices, search engines and IDEs. We may come to wonder how can a device search through a dictionary of thoudsands of words, every instance a key is pressed without lagging. If it would have searched linearly it would become unbearably slow. Additionally, finding words that are common over rarer just because of the alphabetical order creates induces great user experience.

While modern platforms solve this using highly complex neural network and optimized databases, understanding the mechaincs of this means returning to the basics of foundation computer sciences. PACE aims to demonstrate the underlying logic of predictive algorithms as an educational prototype. By building a text engine from scratch using core data structures -Hash Tables for frequency mapping, Binary Search Trees for rapid prefix traversal, and Quick Sort for probabilistic ranking-PACE showcases logical understanding of how memory, search efficiency, and data organization interact to solve real-world bottlenecks.

Motivation behind my project

The motivation behind PACE delved from my curiosity for natural language processing. The 2024 Google autocomplete filtering controversy also played a major role in inducing curiosity about how auto-complete engines not only predict the most probable using frequencies but they can also be manipulated to comply safety filters and business rules.

This encouraged me to at least gain basic knowledge of how predictive text and auto-complete engines works. And I implemented an engine from scratch using Hash Tables and Binary Search Trees, my prototype guarantees deterministic, mathematically pure text prediction where the Quick Sort ranking is determined alone by data frequency only and not any opaque filters.

Objectives:

1. To develop a Hash Table with separate chaining that ingest data and maps frequencies with O(1) time complexity while handling collisions.
2. To construct a Binary Search tree (BST) to organize the trained/ingested vocabulary alphabetically to fetch prefix in O(logn) time.
3. To apply the Quick Sort algorithm to rank the fetched autocomplete suggestions.
4. To design an interactive, menu-driven console application that gracefully handles unpredictable user inputs.

Methodology:

PACE (Predictive Auto-Complete Engine) follows a frequency-driven approach to simulate real-time natural language processing and text prediction. The system utilizes a text corpus (such as a 10,000-word real-world dataset) representing everyday language usage. The individual words within the text are treated as the core data entities, while the number of times each word appears acts as its predictive weight, mimicking how commercial engines learn from user behavior.

The vocabulary database is dynamically constructed in memory, allowing the program to instantly map user inputs to valid words and rank them probabilistically. Three primary data structures and algorithms are applied to process, store, and rank the dataset:

- **Hash Table with Separate Chaining:** Ingests the raw text corpus during the training phase. It maps each word to a generated index, efficiently resolving collisions via linked lists and dynamically updating the frequency count of each unique word with near $O(1)$ time complexity.
- **Binary Search Tree (BST):** Organizes the extracted, unique vocabulary alphabetically. When a user types a few letters, the system traverses the tree to isolate prefix matches in $O(\log n)$ time, completely bypassing the need to scan the entire dictionary.

- **Quick Sort Algorithm:** Determines the final predictive output. It extracts the matched words from the BST and sorts them dynamically based on their learned frequency counts. This ensures that the most statistically probable words are suggested first, overriding standard alphabetical order for a smarter user experience.

Implementation

The project is implemented in the C programming language using an interactive menu-driven approach. The system dynamically processes external text files to build its vocabulary and allows users to test the engine in real-time. The program provides the following operations:

1. **Model Training (Data Ingestion):** Reads an external text file (e.g., a 10,000-word dataset) to ingest vocabulary. It utilizes a Hash Table to identify unique words and tally their frequencies, and then transfers this mapped data into a Binary Search Tree (BST) for hierarchical organization.
2. **Auto-Complete Prediction:** Prompts the user for a text prefix and traverses the BST to locate all matching vocabulary. It then applies the Quick Sort algorithm to an array of these matches, sorting them by historical frequency to output the most statistically probable suggestions.
3. **Dictionary Display:** Utilizes a recursive In-Order tree traversal to output the entire trained dataset to the console, displaying all recognized words alongside their frequency weights in perfect A-to-Z alphabetical order.
4. **Memory Cleanup and Termination:** Employs Post-Order tree traversal and iterative linked-list deletion to safely free all dynamically allocated memory within both the Hash Table and the BST before exiting, ensuring zero memory leaks.
5. **Input Validation & Error Handling:** Secures the menu interface by utilizing standard input buffer clearing techniques (handling invalid scanf inputs) to manage unpredictable user keystrokes gracefully, preventing infinite loops or application crashes during operation.

Working of the System

The program begins by launching an interactive console and displaying a menu of available operations. Users must first initialize the system by supplying a raw text dataset to train the prediction model. During this phase, the engine ingests the text file, calculates exact word frequencies using a Hash Table, and structures the resulting vocabulary into a Binary Search Tree.

Once the model is trained, users can interact with the auto-complete engine by entering various text prefixes to simulate real-world typing. The system instantly traverses the tree to fetch matching vocabulary and applies Quick Sort to rank and display the top suggestions based on their learned statistical probability. Users can also execute a full dictionary display, which performs an In-Order tree traversal to reveal how the entire dataset is stored alphabetically in memory.

This interactive approach allows users to observe the transformation of unstructured data into a rapid predictive model, clearly demonstrating how optimized data structures eliminate computational lag and elevate simple word matching into intelligent, frequency-driven text prediction.

Sr. No.	Hardware/Software	Specification
1	Operating System	Windows 11
2	Hardware	Standard Laptop
3	Programming Language	C programming language
4	IDE/Code Editor	Visual Studio Code
5	Compiler	GCC (GNU Compiler Collection)
6	Data Structures Used	Hash Table (Separate Chaining) and Binary Search Tree (BST)
7	Algorithms Used	Quick Sort, In-Order Traversal, and Post-Order Traversal

Algorithm/Operations Implemented

Data Ingestion Using Hash Table (Separate Chaining)

The system uses a Hash Table to ingest the raw text corpus, identify unique vocabulary, and map word frequencies. Separate chaining via linked lists is utilized to efficiently resolve hash collisions when multiple words map to the same index.

Algorithm:

1. Initialize the hash table array with NULL pointers.
2. Read strings sequentially from the external text corpus.
3. Compute the hash index for each parsed string using a hash function.
4. Traverse the linked list at the computed index to check if the word already exists.
5. If the word exists, increment its frequency count.
6. If the word does not exist, allocate a new node, set its frequency to 1, and insert it into the linked list.

Prefix Organization Using Binary Search Tree (BST)

A Binary Search Tree (BST) is implemented to organize the extracted vocabulary alphabetically. This structure allows the engine to instantly isolate and retrieve words matching a user's typed prefix without scanning the entire dictionary.

Algorithm:

1. Extract unique words and their corresponding frequencies from the Hash Table.
2. Insert each word into the BST by comparing it alphabetically to the current node.
3. Traverse the left subtree if the word is alphabetically lesser, or the right subtree if greater, inserting it as a leaf node when an empty position is found.
4. When a user types a prefix, traverse the tree to locate the first matching node.
5. Perform a localized traversal to extract all descendant nodes that match the prefix string into an array.
6. Stop traversal for branches that fall outside the alphabetical bounds of the prefix.

Probabilistic Ranking Using Quick Sort

Quick Sort is used to dynamically rank the fetched auto-complete suggestions. Instead of sorting alphabetically, it sorts the matched words based on their learned frequency counts, ensuring the most probable words appear at the top of the suggestion list.

Algorithm:

1. Receive the unsorted array of prefix-matched words extracted from the BST.
2. Select a pivot element from the array segment.
3. Partition the array by moving words with higher frequency counts before the pivot, and words with lower frequency counts after the pivot.
4. Recursively apply the pivot selection and partitioning steps to the left and right sub-arrays.
5. Repeat until the entire array segment is sorted.
6. Return the sorted array to the user interface for real-time display.

Executed Code and Output Screenshots

Choice 1

Demo text:

Mini_Project.c | demo.txt

demo.txt

```
I can easily tailor the vocabulary to your exact needs!

Quantum supercomputers seamlessly calculated multidimensional variab
while towering holographic projections illuminated the bustling,
neon-drenched metropolis below. Cybernetic citizens, equipped with a
effortlessly downloaded vast repositories of cosmic data directly in
Meanwhile, sleek starships silently breached the upper stratosphere,
This digital frontier continuously redefines physical boundaries,
blending organic biology with synthetic perfection until reality its

Ancient primordial dragons soared majestically over jagged,
snow-capped peaks, exhaling brilliant sapphire flames that incinerat
forgotten ruins. Beneath the canopy of whispering, enchanted woodlan
ethereal elven sorcerers channeled mystical currents to fortify thei
Valorous knights, clad in impenetrable runic armor, brandished leger
Destiny beckoned these disparate factions, forging an unbreakable al


Sophisticated computational algorithms systematically manipulate com
Programmers deftly implement dynamic arrays, interconnected linked l
while utilizing robust hash tables for near-instantaneous key-value
complementing graph traversal techniques such as depth-first search
scalable software engineering.
```

```
PS C:\Users\aakef\Desktop\DS Mini Project> gcc .\Mini_Project.c
PS C:\Users\aakef\Desktop\DS Mini Project> .\a.exe

=== NLP Auto-Complete Engine ===
1. Train Model from Text File
2. Test Auto-Complete Prediction
3. Display Dictionary
4. Exit
Enter your choice: 1
Enter training filename (e.g., training_data.txt): demo.txt
Training AI model...
Training complete! Dictionary organized.
```

Choice 2

```
=== NLP Auto-Complete Engine ===
1. Train Model from Text File
2. Test Auto-Complete Prediction
3. Display Dictionary
4. Exit
Enter your choice: 2
Enter a prefix to auto-complete: mom

--- Auto-Complete Suggestions ---
1. moment (Frequency: 3)
2. moments (Frequency: 2)
--------------------------------

=== NLP Auto-Complete Engine ===
1. Train Model from Text File
2. Test Auto-Complete Prediction
3. Display Dictionary
4. Exit
Enter your choice: 2
Enter a prefix to auto-complete: com

--- Auto-Complete Suggestions ---
1. completely (Frequency: 2)
2. complex (Frequency: 2)
3. coming (Frequency: 1)
--------------------------------
```

Choice 3

```
=== NLP Auto-Complete Engine ===
1. Train Model from Text File
2. Test Auto-Complete Prediction
3. Display Dictionary
4. Exit
Enter your choice: 3

--- Full Alphabetical Dictionary ---
Word: (a                | Frequency: 1
Word: (existential      | Frequency: 1
Word: (resplendent,     | Frequency: 1
Word: a                 | Frequency: 23
Word: about             | Frequency: 5
Word: absolute          | Frequency: 1
Word: across            | Frequency: 1
Word: actually          | Frequency: 1
Word: adjective,        | Frequency: 1
Word: advanced          | Frequency: 1
Word: adverb            | Frequency: 1
Word: after             | Frequency: 3
Word: afternoon         | Frequency: 1
Word: against           | Frequency: 2
Word: ahead,            | Frequency: 1
```

Choice 4

```
=== NLP Auto-Complete Engine ===
1. Train Model from Text File
2. Test Auto-Complete Prediction
3. Display Dictionary
4. Exit
Enter your choice: 4
Cleaning up memory...
Memory completely freed. Exiting Engine. Goodbye!
PS C:\Users\aakef\Desktop\DS Mini Project>
```

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
2. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
3. Sedgewick, R., & Wayne, K. (n.d.). *Hash tables*. Algorithms, 4th Edition. Princeton University. https://algs4.cs.princeton.edu/34hash/
4. Sedgewick, R., & Wayne, K. (n.d.). *Binary search trees*. Algorithms, 4th Edition. Princeton University. https://algs4.cs.princeton.edu/32bst/
5. Sedgewick, R., & Wayne, K. (n.d.). *Quicksort*. Algorithms, 4th Edition. Princeton University. https://algs4.cs.princeton.edu/23quicksort/
6. Google Books. (2012). *Google Books Ngram Viewer dataset*. Google Books. https://books.google.com/ngrams
7. Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.